

Name: Charmi Gaddale

Roll Number: 23071A0516

AIM: Implement unsupervised k- means algorithm

Dataset used: House Price Prediction dataset

```
# Import required libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# For scaling the data
from sklearn.preprocessing import StandardScaler

# Clustering algorithms
from sklearn.cluster import KMeans
from sklearn_extra.cluster import KMedoids

# Evaluation metric
from sklearn.metrics import silhouette_score
```

```
from google.colab import files
uploaded = files.upload()
```

[Choose files](#) Housing.csv

Housing.csv(text/csv) - 29981 bytes, last modified: 19/04/2026 - 100% done
Saving Housing.csv to Housing (2).csv

```
# Read dataset
df = pd.read_csv("Housing.csv")

# Display first 5 rows
df.head()
```

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	hotwaterheating	airconditioning	parking	pr
0	13300000	7420	4	2	3	yes	no	no	no	yes	2	
1	12250000	8960	4	4	4	yes	no	no	no	yes	3	
2	12250000	9960	3	2	2	yes	no	yes	no	no	2	
3	12215000	7500	4	2	2	yes	no	yes	no	yes	3	
4	11410000	7420	4	1	2	yes	yes	yes	no	yes	2	

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
# Select only area and price for clustering
data = df[['area', 'price']]

data.head()
```

	area	price
0	7420	13300000
1	8960	12250000
2	9960	12250000
3	7500	12215000
4	7420	11410000

Next steps: [Generate code with data](#) [New interactive sheet](#)

```
# Standardize the features
scaler = StandardScaler()

scaled_data = scaler.fit_transform(data)

scaled_data[:5]
```

```
array([[1.04672629, 4.56636513],
       [1.75700953, 4.00448405],
       [2.21823241, 4.00448405],
       [1.08362412, 3.98575468],
       [1.04672629, 3.55497918]])
```

```
scaler = StandardScaler()

scaled_data = scaler.fit_transform(data)

scaled_data[:5]
```

```
array([[ 4.56636513,  1.04672629,  1.40341936,  1.42181174,  1.37821692,
         1.51769249],
       [ 4.00448405,  1.75700953,  1.40341936,  5.40580863,  2.53202371,
         2.67940935],
       [ 4.00448405,  2.21823241,  0.04727831,  1.42181174,  0.22441013,
         1.51769249],
       [ 3.98575468,  1.08362412,  1.40341936,  1.42181174,  0.22441013,
         2.67940935],
       [ 3.55497918,  1.04672629,  1.40341936, -0.57018671,  0.22441013,
         1.51769249]])
```

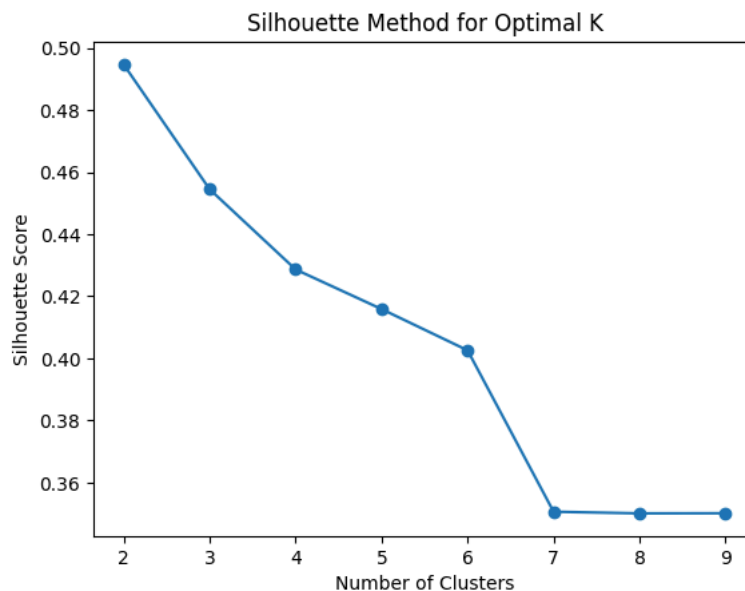
```
# Try different number of clusters
scores = []
k_values = range(2,10)

for k in k_values:

    kmeans = KMeans(n_clusters=k, random_state=0)
    labels = kmeans.fit_predict(scaled_data)

    score = silhouette_score(scaled_data, labels)
    scores.append(score)

# Plot silhouette scores
plt.plot(k_values, scores, marker='o')
plt.xlabel("Number of Clusters")
plt.ylabel("Silhouette Score")
plt.title("Silhouette Method for Optimal K")
plt.show()
```



```
# Apply KMeans using optimal k = 2
kmeans = KMeans(n_clusters=2, random_state=0)

# Fit the model and assign cluster labels
df["KMeans_Label"] = kmeans.fit_predict(scaled_data)

df.head()
```

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	hotwaterheating	airconditioning	parking	pr
0	13300000	7420	4	2	3	yes	no	no	no	yes	2	
1	12250000	8960	4	4	4	yes	no	no	no	yes	3	
2	12250000	9960	3	2	2	yes	no	yes	no	no	2	
3	12215000	7500	4	2	2	yes	no	yes	no	yes	3	
4	11410000	7420	4	1	2	yes	yes	yes	no	yes	2	

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
# Apply KMedoids clustering
kmedoids = KMedoids(n_clusters=2, random_state=0)

# Fit the model
df["KMedoids_Label"] = kmedoids.fit_predict(scaled_data)

df.head()
```

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	hotwaterheating	airconditioning	parking	pr
0	13300000	7420	4	2	3	yes	no	no	no	yes	2	
1	12250000	8960	4	4	4	yes	no	no	no	yes	3	
2	12250000	9960	3	2	2	yes	no	yes	no	no	2	
3	12215000	7500	4	2	2	yes	no	yes	no	yes	3	
4	11410000	7420	4	1	2	yes	yes	yes	no	yes	2	

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
# Evaluate clustering performance
kmeans_score = silhouette_score(scaled_data, df["KMeans_Label"])
kmedoids_score = silhouette_score(scaled_data, df["KMedoids_Label"])

print("KMeans Silhouette Score:", kmeans_score)
print("KMedoids Silhouette Score:", kmedoids_score)
```

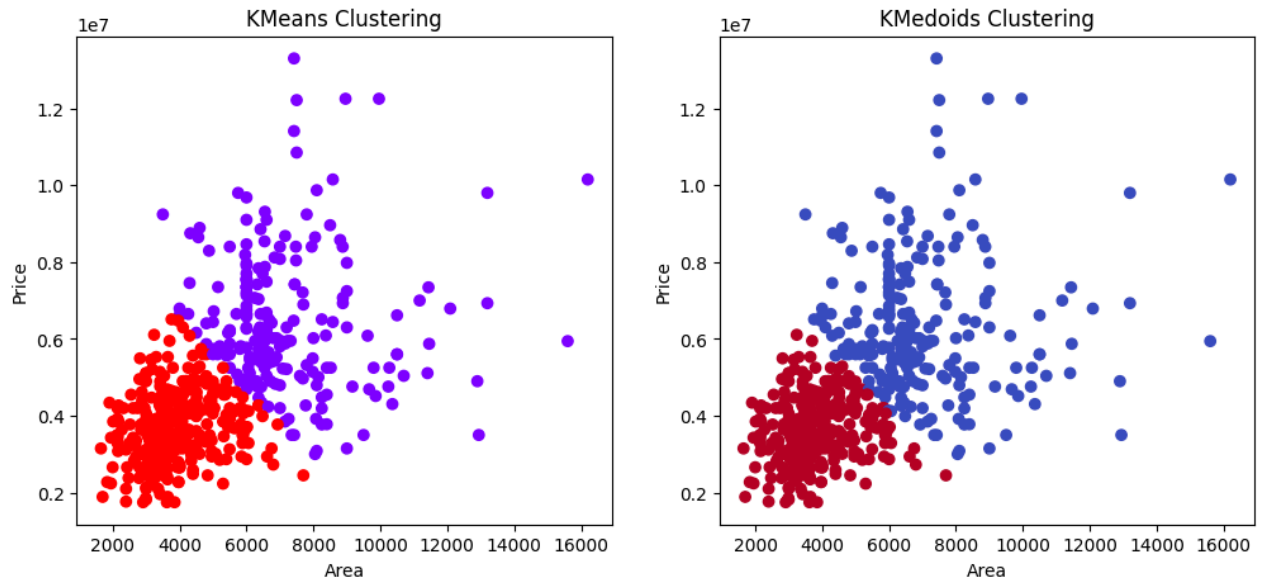
KMeans Silhouette Score: 0.49468992381056137
KMedoids Silhouette Score: 0.48639666907446816

```
plt.figure(figsize=(12,5))

# KMeans
plt.subplot(1,2,1)
plt.scatter(df["area"], df["price"], c=df["KMeans_Label"], cmap="rainbow")
plt.title("KMeans Clustering")
plt.xlabel("Area")
plt.ylabel("Price")

# KMedoids
plt.subplot(1,2,2)
plt.scatter(df["area"], df["price"], c=df["KMedoids_Label"], cmap="coolwarm")
plt.title("KMedoids Clustering")
plt.xlabel("Area")
plt.ylabel("Price")
```

```
plt.show()
```



Inference: K-Means and K-Medoids clustering algorithms were applied to the housing dataset using area and price as features. The data was standardized using StandardScaler to ensure both features contributed equally to the clustering process. The optimal number of clusters was determined using the Silhouette Method, and the highest silhouette score was obtained at $k = 2$, indicating that two clusters provide the best separation for the dataset. The silhouette score obtained was approximately 0.49, which indicates a reasonable clustering structure. The clustering results show that houses with similar area and price values are grouped together, forming distinct clusters. Visualization of the clusters confirms that the dataset is divided into two meaningful groups based on housing characteristics.